FluxCD v2 + SOPS

Secrets Management Lab Guide

Encrypt, Commit, and Decrypt Kubernetes Secrets Securely with GitOps

4 Labs	20+ Commands	Production-Ready

Table of Contents

Lab 1 — SOPS & age Setup ····· Install SOPS, generate age keys, configure encryption
Lab 2 — Encrypt & Commit Secrets ····· Encrypt Kubernetes secrets with SOPS and push to GitLab
Lab 3 — Flux Decryption Integration ····· Configure Flux to auto-decrypt secrets at apply time
Quick Reference Cheat Sheet

LAB 1 — SOPS & age Setup

Objective: Install SOPS and the age encryption tool, generate your age key pair, and configure SOPS to use age for encrypting Kubernetes secrets.

Estimated Time

30 minutes

Step 1

Install SOPS CLI

Choose the installation method for your operating system.

Linux (x86_64):

```
curl -LO https://github.com/getsops/sops/releases/download/v3.12.1/sops-v3.12.1.linux.amd64

# Move the binary in to your PATH
mv sops-v3.12.1.linux.amd64 /usr/local/bin/sops

# Make the binary executable
chmod +x /usr/local/bin/sops
```

Step 2

Install age

age is a modern, simple encryption tool — the recommended backend for SOPS.

Linux:

```
wget https://github.com/FiloSottile/age/releases/download/v1.3.1/age-v1.3.1-linux-amd64.tar.gz
tar -zxvf age-v1.3.1-linux-amd64.tar.gz
cd age
cp age* /usr/local/bin
age --version
```

Step 3

Generate an age Key Pair

Your age private key decrypts secrets. Keep it out of Git — always. Don't run in fleet directory

```
# Generate the key pair (outputs to stdout)
age-keygen -o age.agekey
```

```
# Expected output:
# Public key: age1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

# Display both keys
cat age.agekey
# AGE-SECRET-KEY-... (private key - NEVER commit this)
# public key: age1...   (share this — used in .sops.yaml)

# Store the public key in an env var for convenience
export AGE_PUBLIC_KEY=$(grep 'public key:' age.agekey | awk '{print $NF}')
echo $AGE_PUBLIC_KEY
```

Key Security Rules

- NEVER commit age.agekey or any private key to Git
- Add age.agekey to your .gitignore immediately
- Back up your private key to a secure vault (e.g., 1Password, HashiCorp Vault)
- In CI/CD pipelines, inject the private key via environment variables or Vault

Step 4 — Create a .sops.yaml Configuration

Define which files SOPS encrypts and which age public key to use.

In the root of your fleet-infra repository:

```
# fleet-infra1/.sops.yaml — commit this file to Git
cat > .sops.yaml <<EOF
creation_rules:
  - path_regex: .*/secrets/.*\.yaml$
    encrypted_regex: ^(data|stringData)$
    age: >-
      ${AGE_PUBLIC_KEY}
EOF

# Commit the config (NOT the private key)
git add .sops.yaml .gitignore
git commit -m 'feat: add SOPS encryption config'
git push
```

✓ SOPS CLI installed and verified
✓ age installed — age-keygen available
✓ age key pair generated — public key exported to $AGE_PUBLIC_KEY
✓ .sops.yaml committed to fleet-infra — encryption rules defined
✓ Private key secured and excluded from Git via .gitignore

LAB 2 — Encrypt & Commit Secrets

Objective: Create a Kubernetes Secret manifest, encrypt it with SOPS using your age key, and safely push it to GitLab.

- Create a raw Kubernetes Secret YAML
- Encrypt it in-place with SOPS (age backend)
- Understand what the encrypted file looks like
- Safely commit and push encrypted secrets to GitLab

30 minutes

Step 1

Create the Directory Structure

Organize your secrets in a dedicated path that matches your .sops.yaml regex.

```
# Inside your fleet-infra repo
mkdir -p clusters/my-cluster/secrets

# The .sops.yaml path_regex matches: .*/secrets/.*\.yaml$
# Any YAML file inside a 'secrets' directory will be auto-encrypted
```

Step 2

Create a Raw Kubernetes Secret

Write the plaintext secret manifest before encryption.

```
mkdir clusters/my-cluster/secrets
cat > clusters/my-cluster/secrets/db-credentials.yaml <<EOF
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
  namespace: default
type: Opaque
stringData:
  DATABASE_URL: postgresql://admin:super-secret-
password@db.example.com:5432/mydb
  DATABASE_PASSWORD: super-secret-password
  API_KEY: sk-prod-abc123xyz789
EOF
```

⚠ Never Commit This File Unencrypted

At this point db-credentials.yaml is plaintext. Do NOT run git add yet.
Always encrypt before committing. The next step encrypts the file in-place.

Step 3 — Encrypt with SOPS

Run sops --encrypt to encrypt the secret values in-place.

```
# Encrypt in-place using the age public key from .sops.yaml
sops --encrypt --in-place clusters/my-cluster/secrets/db-credentials.yaml

# Verify the file is now encrypted
cat clusters/my-cluster/secrets/db-credentials.yaml
```

After encryption, the file will look like this:

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
  namespace: default
type: Opaque
stringData:
  DATABASE_URL: ENC[AES256_GCM,data:xxxxxxxxxxxxx,tag:yyy,type:str]
  DATABASE_PASSWORD: ENC[AES256_GCM,data:zzzzzzzzzzz,tag:aaa,type:str]
  API_KEY: ENC[AES256_GCM,data:bbbbbbbbbbb,tag:ccc,type:str]
sops:
  kms: []
  age:
    - recipient: age1xxxxxxxxxxxxxxxxxxxxxxxxx
      enc: |
        -----BEGIN AGE ENCRYPTED FILE-----
        xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
        -----END AGE ENCRYPTED FILE-----
  version: 3.x.x
```

Step 4 — Edit an Encrypted Secret

Use sops (without --encrypt) to open, edit, and re-save an encrypted file.

```
# SOPS decrypts, opens your $EDITOR, then re-encrypts on save
export SOPS_AGE_KEY_FILE=path of the key /age.agekey
sops clusters/my-cluster/secrets/db-credentials.yaml

# Change a value, save, and SOPS re-encrypts automatically
# NEVER manually edit the ENC[...] ciphertext
```

Step 5 — Commit & Push to GitLab

The encrypted file is safe to commit — no secrets are exposed.

```
git add clusters/my-cluster/secrets/db-credentials.yaml
git commit -m 'feat: add encrypted db-credentials secret'
git push
```

```
# Verify the commit contains only encrypted values
git show HEAD:clusters/my-cluster/secrets/db-credentials.yaml | grep DATABASE
```

LAB 3 — Flux Decryption Integration

Objective: Configure Flux's Kustomize controller to automatically decrypt SOPS-encrypted secrets when applying them to your Kubernetes cluster.

What You Will Learn

- Store your age private key as a Kubernetes Secret in flux-system
- Configure a Kustomization resource with decryption settings
- Verify Flux creates the decrypted Kubernetes Secret in the cluster
- Understand the full GitOps secrets lifecycle

60 min

How Flux SOPS Decryption Works

- Flux source-controller pulls the encrypted secret YAML from GitLab
- Flux kustomize-controller reads the sops.decryption config on the Kustomization
- Controller retrieves the age private key from the sops-age Kubernetes Secret
- SOPS library decrypts the ENC[...] values in memory
- kustomize-controller applies the decrypted Secret to the cluster
- The plaintext private key NEVER touches disk or Git — only lives in cluster memory

Step 1

Store the age Private Key in the Cluster

Create a Kubernetes Secret containing your age private key in the flux-system namespace.

```
# Create the Secret from your age private key file

kubectl create secret generic sops-age  --namespace=flux-system --from-
file=age.agekey=/home/labuser/age/age.agekey

# Verify the secret was created
kubectl get secret sops-age -n flux-system

# Confirm it contains the key (value will be base64-encoded)
kubectl get secret sops-age -n flux-system -o jsonpath='{.data.age\.agekey}' |
base64 -d | head -2
```

Why kubectl create and not Git?

This secret contains your PRIVATE key — it must NEVER be in Git.
Creating it with kubectl create secret means it lives only in the cluster's etcd, not in any repository.
In production, use an external secrets operator or CI/CD pipeline to inject this secret.

Step 2 — Create a Kustomization with Decryption

Add a decryption block to your Kustomization resource pointing to the sops-age secret.

```
# clusters/my-cluster/secrets-kustomization.yaml
cat > clusters/my-cluster/secrets-kustomization.yaml <<EOF
---
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: secrets
  namespace: flux-system
spec:
  interval: 5m
  path: ./clusters/my-cluster/secrets
  prune: true
  sourceRef:
    kind: GitRepository
    name: flux-system
  decryption:
    provider: sops
    secretRef:
      name: sops-age
  healthChecks:
    - apiVersion: v1
      kind: Secret
      name: db-credentials
      namespace: default
EOF
```

Step 3 — Commit and Push the Kustomization

Pushing triggers Flux to pick up the new Kustomization and start decrypting secrets.

```
git add clusters/my-cluster/secrets-kustomization.yaml
git commit -m 'feat: add secrets Kustomization with SOPS decryption'
git push

# Force immediate reconciliation
flux reconcile kustomization flux-system --with-source
```

Step 4 — Watch Flux Reconcile the Secret

Monitor Flux as it detects, decrypts, and applies your secret.

```
# Watch the secrets Kustomization
flux get kustomizations --watch

# Expected output:
# NAME     REVISION    SUSPENDED  READY  MESSAGE
# secrets  main/abc123 False      True   Applied revision: main/abc123

# Verify the Secret was created in the cluster
kubectl get secret db-credentials -n default

# Confirm the values were decrypted correctly
kubectl get secret db-credentials -n default -o
jsonpath='{.data.DATABASE_PASSWORD}' | base64 -d
```

Step 5

View Flux Events for Secret Reconciliation

Inspect events to confirm successful decryption.

```
# See Flux events for the secrets Kustomization
flux events --for Kustomization/secrets

# Check the kustomize-controller logs for decryption activity
kubectl logs -n flux-system -l app=kustomize-controller -f | grep -i sops

# Expected log line:
# {"level":"info","msg":"decrypted 1 Secret(s) using sops provider"}
```